QUESTIONSSSSSSS:

Category 1: Algorithmic Engine & Optimization (The Hardest Technical Questions)
Because your core feature relies on a multi-objective optimization workflow, the technical panelists will focus heavily on how you handle your mathematical boundaries.

1. **"Since academic timetabling is an NP-hard problem, how exactly does your Hybrid Algorithm navigate the constraint space without crashing?"**
    - ***Targeted Answer:*** **Explain that a standard exhaustive search triggers a combinatorial explosion. Your Greedy deterministic constructor plots a fast, baseline allocation in milliseconds, and the Genetic Algorithm (GA) uses biological crossover and mutation to shuffle choices and eliminate remaining bottlenecks.**
2. **"Why use a Hybrid Greedy-Genetic Algorithm? Why not just use Tabu Search or Particle Swarm Optimization (PSO), which are standard in literature?"**
    - ***Targeted Answer:*** **Tabu Search relies entirely on local, single swaps, which easily get trapped in "local optima" dead-ends when dealing with the dense room limits of local public schools. PSO is natively designed for continuous fluid variables (like physics calculation) and requires clunky conversion wrappers to handle discrete, rigid data variables like class placements.**
3. **"How do you mathematically distinguish a Hard Constraint from a Soft Constraint in your backend code?"**
    - ***Targeted Answer:*** **Hard constraints are non-negotiable validations built as absolute rules (e.g., stopping double-bookings or matching a TLE subject to a workshop room). Soft constraints are processed using your Configurable Priority Weights (Objective 1.2); the GA evaluates them using a *fitness function* that assigns higher quality scores to schedules that fulfill things like teacher travel-time preferences.**
4. **"What happens if the inputs are physically impossible? For example, if there are 30 sections but only 10 available physical classrooms?"**
    - ***Targeted Answer:*** **As proven in your Test Case 08 (TC-08), the algorithm detects an incomplete or contradictory dataset, halts generation, and outputs an error message highlighting the exact spatial limitation rather than generating a corrupted schedule.**
5. **"If your system is a 'Decision-Support Web Application', why do you need a manual drag-and-drop refinement interface if the algorithm is supposed to be automated?"**
    - ***Targeted Answer:*** **An algorithm cannot always anticipate dynamic, real-world scenarios or changing school requirements. Automated generation provides a mathematically perfect baseline, but your Objective 1.1 drag-and-drop tool leaves the ultimate decision-making control with the human scheduler.**

Category 2: Database Architecture & Integrity (The 28-Table Architecture)
Your manuscript reveals a comprehensive database layer. The panel will look at your Entity Relationship Diagram (ERD) to verify your data normalization and synchronization design.

6. **"Your system utilizes a 28-table architecture. Why does an academic scheduling tool require so many tables?"**
    - ***Targeted Answer:*** **To prevent data duplication and protect operational tracking. Separate data stores are required for global rules (scheduling_policies), audit logs (audit_logs), change histories (manual_schedule_edits), and independent preference sub-intervals (preference_time_slots).**
7. **"What are your 'Teacher Mirrors' (teacher_mirrors) and 'Section Mirrors' (section_mirrors) tables? Why are they called mirrors?"**
    - ***Targeted Answer:*** **They act as local, static snapshots of external student data platforms (like the DepEd LIS system). Schedulers synchronize data to these mirror tables to ensure that any network connection gaps do not disrupt the live central algorithm runtime.**
8. **"How does the database handle the 'Locked Sessions' (locked_sessions) feature during manual refinement?"**
    - ***Targeted Answer:*** **When a scheduler manually moves a class to a specific time or room and 'pins' it, that allocation is saved to the locked_sessions table. During subsequent automated runs, the constructor respects these rows as frozen hard parameters that cannot be overwritten.**
9. **"How do you secure teacher data, specifically regarding passwords and permission elevations?"**
    - ***Targeted Answer:*** **Account credentials undergo bcrypt password hashing and access control is guarded via JWT-based authentication, separating role-based privileges between the System Administrator, Academic Scheduler, Teachers, and view-only Students.**

Category 3: Infrastructure, Caching & Connectivity (LAN vs. WAN)

The panel will focus heavily on how you deliver your unique value proposition: being online-first but offline-capable.

10. **"Explain exactly how A.T.L.A.S. functions as an 'online-first but offline-capable' application."**
    - ***Targeted Answer:*** **High-dependency operations—like teachers submitting preferences or publishing new schedule modifications—require an active network path to talk to the database. However, on-campus users can still access the system fully using the school LAN even if public internet lines drop.**
11. **"How does 'smart client-side caching' work when a student or teacher goes completely offline outside the school network?"**
    - ***Targeted Answer:*** **When a remote user loses connection, the application avoids throwing a standard connection error. Instead, it safely loads the last viewed schedule state directly from the device's local memory cache.**
12. **"How does 'automatic background synchronization' function when the network returns?"**
    - ***Targeted Answer:*** **Any preference forms or administrative adjustments completed during a connection gap are temporarily preserved locally. The exact moment connection is restored, a background synchronization mechanism pushes those updates to the main server database and refreshes the live dashboards seamlessly.**
13. **"Why did your group select a localized, self-hosted on-premises deployment instead of standard cloud hosting like AWS or a VPS?"**
    - ***Targeted Answer:*** **On-premises self-hosting (using an Ubuntu Linux server and a local PostgreSQL database) gives the institution complete control over its information data while eliminating the financial burden of recurring commercial cloud subscriptions.**

Category 4: Research Methodology & Evaluation Standards (The Capstone Requirements)

Your panel will check if your actual engineering results match the goals set during your descriptive phase.

14. **"Your study utilizes a combined Descriptive and Developmental Research Design. How do these two separate approaches relate to each other?"**
    - ***Targeted Answer:*** **Your Descriptive phase acted as a qualitative needs assessment to capture manual constraints at Hinigaran National High School. Those real-world observations directly provided the foundational logic needed to systematically design, engineer, and validate your Developmental intervention.**
15. **"In your Sampling Technique discussion, you mention using a quantitative approach, but your needs assessment was qualitative. Is there an internal conflict here?"**
    - ***Targeted Answer:*** **No, because your project follows two distinct sequential stages. Your *requirements gathering* was qualitative (interviews/observations), but your *system evaluation phase* is strictly quantitative to ensure objective, statistically valid data.**
16. **"How does your Test Case Questionnaire utilize the AAA (Arrange, Act, Assert) software development standard?"**
    - ***Targeted Answer:*** **Rather than using a loose checklist, every test case strictly configures the input data (Arrange), executes the core system process (Act), and verifies that the system output matches your explicit criteria (Assert), testing both expected user actions and invalid inputs.**
17. **"How will you translate your 5-point Likert scale results from the ISO/IEC 25010 questionnaire into actionable engineering conclusions?"**
    - ***Targeted Answer:*** **Aggregated scores are averaged first per sub-characteristic to locate specific technical software vulnerabilities. Then, they are grouped into primary characteristics to compute the final deployment readiness score of the application.**

💡 **Pro-Defense Strategy Checklist for Your Team:**

- **Keep Your Answers Domain-Specific: When discussing technical objectives, always ground your answer by naming the specific pilot institution (Hinigaran National High School) to emphasize practical deployment.**
- **Avoid Reading Text: Place the academic validation or research names (e.g., *Han & Wang, 2025* or *Sun & Wu, 2023*) directly on the slide as visual proof, while using your conversational tone to discuss the technical concept and gap address.**
- **Reference Your DFD Clusters: If a panelist questions how data moves from a teacher's phone to the algorithm, immediately point to your DFD slides: Cluster A handles user access, Cluster B captures parameters, and Cluster C runs the optimization engine.**

PROMPTTTTTTTTTTT:

Issue: The **"Close Profile"** button at the bottom of the modal currently uses a light gray background with gray text.
UX/UI Impact: In modern UI/UX design patterns, this specific grayed-out styling strongly signals a **disabled ("unabled")** state. Users may mistakenly believe the button is unclickable or that they are blocked from closing the profile modal, leading to confusion and a poor user experience.
Recommendation for Implementation:

- **Update Button Styling:** Change the background and text color of the "Close Profile" button to indicate it is an active, secondary action.
    - **Design Suggestion:** Use a clear outline (ghost button style) matching the system's primary color theme, or a distinct text color that contrasts well against the light background to show interactivity

Context: On the **Faculty** list view, primary actions—"View Assigned Classes" (user icon) and "Manage Teaching Load" (document/list icon)—are explicitly exposed as standalone action buttons in the table row. However, these exact same actions are duplicated inside the **Three Dots (Ellipsis) More Actions menu** to their right.
Decision: To streamline the interface and drastically reduce the number of clicks required for user tasks, the decision is to **keep the explicit standalone action icons** and completely **remove the three-dots menu**.
Task for Sections Navigation Menu: Apply this exact same UX optimization logic to the **Sections** view.

- Identify if primary actions are duplicated between standalone row buttons and a "three dots" ellipsis menu.
- **Remove the redundant three-dots menu icon** entirely from the actions column.
- Ensure the primary action icons remain visible and directly clickable in each row to maintain quick, low-click workflows across the system.

Issue: The system currently uses informal or abbreviated text for UI labels and headers in several places (e.g., using **"DEPT"** instead of the full word **"Department"** in section headers like *IDENTITY & DEPT*).
Goal: Improve system professionalism, readability, and UI text consistency by ensuring all conversational or functional labels use fully spelled-out words.
Action Items for the Agent:

1. **Global Codebase Scan:** Scan all UI text strings, table headers, sidebar navigation menus, and card components across the entire system for unnecessary abbreviations.
2. **Text Replacements:** Replace identified abbreviations with their complete, formal terms.
    - *Example:* Convert DEPT or Dept $\rightarrow$ Department
    - *Example:* Convert S.Y. $\rightarrow$ School Year *(if applicable to project standards)*
3. **Strict Scope Exclusions:** Do **not** expand structural academic codes or database keys. Leave the following exactly as they are:
    - **Subject Codes:** (e.g., AP, SCI_ES, TLE, ESP, TLE_ICT_EXP)
    - **Technical Identifiers / Short Identifiers**

Issue: The system currently displays grade levels using a short **"G + Number"** format (e.g., G7, G8, G9, G10) within UI badges and tags.
Goal: Standardize the naming convention across the entire application to use the more formal and explicit **"GR + Number"** format to improve clarity and professional presentation.
Action Items for the Agent:

1. **Global UI Scan:** Search all code components, utility files, or badge renderers that format or display student/faculty grade levels in the front-end interface.
2. **Tag Format Migration:** Update the rendering text strings to map to the new **"GR"** prefix:
    - Rename G7 -> GR7
    - Rename G8 -> GR8
    - Rename G9 -> GR9
    - Rename G10 -> GR10
3. **Scope Check:** Ensure this text replacement updates all visual badges, filtering dropdowns, and list views across all system navigation modules (Faculty, Sections, Timetable, etc.) where these grade tags appear.

Module: Campus & Rooms Map Editor (/school-setup/campus-rooms)

Task 1: Missing "Remove Background" / Reset Functionality

- **Issue:** While users can upload and change the custom map background image using the **"Background"** action button, there is currently no way to completely remove it or revert back to the default plain grid canvas.
- **Requirement:** Add a "Remove Background" or "Clear Image" feature.
  - *Option A:* Clicking the "Background" button should present a dropdown/modal option to "Remove Current Background."
  - *Option B:* Wire the existing **"Reset View"** button to also clear any custom uploaded background image, returning the canvas layout to its default plain, empty state.

Task 2: Dynamic Canvas Resizing / Auto-Expanding Background

- **Issue:** The interactive canvas bounding box (or the background layout container) has fixed dimensions. As shown in the UI, building blocks (colored canvas nodes) placed on the outer edges are overflowing and extending past the borders of the visual workspace background.
- **Requirement:** Implement a dynamic bounding box or auto-expanding background dimension calculation.
  - **Logic:** The layout canvas's minimum total width and height should automatically calculate and adapt based on the maximum coordinates of the placed building objects.
  - **Formula Idea:**

Formula Idea:

$$\text{Canvas Width} = \max(X_n + \text{Width}_n) + \text{Padding}$$

$$\text{Canvas Height} = \max(Y_n + \text{Height}_n) + \text{Padding}$$

  - **Expected Behavior:** Ensure no colored building box ever overflows or clips past the viewable background area, regardless of how far right or down a user moves a building block.

Module: Main Dashboard Canvas (/dashboard)

The Problem: High Data Density & Text Clipping

- **Issue:** The persistent right-hand sidebar is too narrow to handle detailed building analytics and floor plans. As a result, critical information under the **Floor Plan** section is heavily compressed, text is cut off (truncated), and the font size is too small to read comfortably.
- **Impact:** Poor readability and cluttered layout on initial page load, degrading the primary dashboard experience.

The Proposed Solution: Interactive "Click-to-Reveal" Drawer Flow

1. Initial Dashboard State (On First Load)

- **Behavior:** Hide the right-hand panel entirely by default.
- **Layout:** Give the **Campus Map** grid canvas (or the expanded Building / Floor Plan vector layout) full-width priority across the screen space. This gives the core interactive map room to breathe.

2. Contextual Drawer Trigger

- **Behavior:** When a user explicitly **clicks** on a specific building block or a particular room node within the canvas, trigger an **animated sliding drawer** that emerges from the right side of the viewport.
- **Layout Advantage:** By utilizing a slideout drawer instead of a cramped persistent column, we can allocate a much larger width (e.g., $400\text{px} - 500\text{px}$ or a responsive width fraction) to display full room codes, capacity gauges, occupancy indicators, and conflict warnings without text truncation.

3. Drawer Controls

- Ensure the drawer includes a clear **dismiss/close icon (X)** at the top right, clicking backdrop areas closes it, or deselecting the building hides the drawer and returns the canvas to full-width view.

Module: Global Sidebar Navigation (Sidebar / NavigationLayout component)

The Problem: High Cognitive Load & Friction in Minimized State

- **Current Behavior:** The navigation sidebar operates on a strict click-to-toggle basis between its expanded and minimized (icon-only) states.
- **User Friction:** When the sidebar is minimized, users who have not yet memorized the system icons must repeatedly guess or manually toggle the bar open just to read the menu labels. This drastically increases task completion time and cognitive fatigue.

The Proposed Solution: Dual-Mode Dynamic Sidebar (Hover-Preview + Click-to-Lock)

We want to implement a fluid, intuitive hover behavior that allows quick scanning without forcing users to permanently toggle the screen layout.

1. "Hover-to-Preview" Behavior (When Minimized)

- **Trigger:** When the sidebar is minimized and the user **hovers (onMouseEnter)** their cursor anywhere over the sidebar container.
- **Action:** Smoothly transition the sidebar to its **fully expanded state** (showing both icons and text labels) as a temporary overlay *without* shifting or resizing the main dashboard content layout.
- **Dismissal:** When the cursor leaves the sidebar area **(onMouseLeave)**, it automatically collapses back to the compact, icon-only view.

2. "Click-to-Lock" Behavior (Sticky State)

- **Trigger:** Clicking the explicit hamburger/sidebar toggle button.
- **Action:** Permanently locks the sidebar into the fully expanded state. When locked, the main dashboard content area should shrink horizontally to accommodate the fixed sidebar width. Hovering off the panel in this state will **not** collapse it.

Technical Implementation Notes for the Agent:

- Manage this using a combined state machine or React component state variables (e.g., isLocked and isHovered).
- Apply a CSS transition rule (e.g., transition: width 0.3s ease-in-out) to make the expansion feel fluid rather than abrupt.
- When expanding via hover, ensure the sidebar uses a high z-index so it floats neatly over the dashboard workspace without causing layout shifts or pushing the main cards out of place.

Module: Faculty Planning / Teachers List (/teachers) and global avatar rendering components.

The Problem: Mismatched Avatar Initials

- **Issue:** The system dynamically generates two-letter text initials inside circular profile avatars, but the generation logic is currently pulling characters in the wrong order.
- **Example from UI:** For the user **AQUINO, ELPIDIO**, the avatar displays **"EA"** instead of **"AE"**.
- **Root Cause:** The logic is reading the first letter of the *First Name* (E) followed by the first letter of the *Last Name* (A). However, because the system's text display format strictly presents the name as **[Surname], [First Name]**, the visual reading order is broken, making the initials feel completely disconnected from the text beside them.

Action Items for the Agent:

1. **Refactor Initial Generation Logic:**
   Update the avatar string parser to accurately respect the primary display sequence of the text (Surname first).
   - **Current string parsing behavior:** Initials = FirstName[0] + LastName[0]
   - **Expected string parsing behavior:** Initials = LastName[0] + FirstName[0]
2. *Target Mapping Examples:*
   - AQUINO, ELPIDIO $\rightarrow$ Change from **EA** to **AE**
   - AQUINO, FERDINAND $\rightarrow$ Change from **FA** to **AF**
   - AQUINO, GRACE $\rightarrow$ Change from **GA** to **AG**
3. **Global Codebase Scan:**
   Identify the reusable Avatar component (e.g., <Avatar />, <UserCircle />, or custom initials helper functions) and apply this algorithmic fix globally. Ensure this corrected layout pattern reflects consistently across the Dashboard, Faculty tables, Profile headers, and audit logs.